



清华大学 交叉信息研究院
Institute for Interdisciplinary Information Sciences, Tsinghua University



Advanced Topics for Robotics 智能机器人前沿探究

Lecture 7: Robot Perception

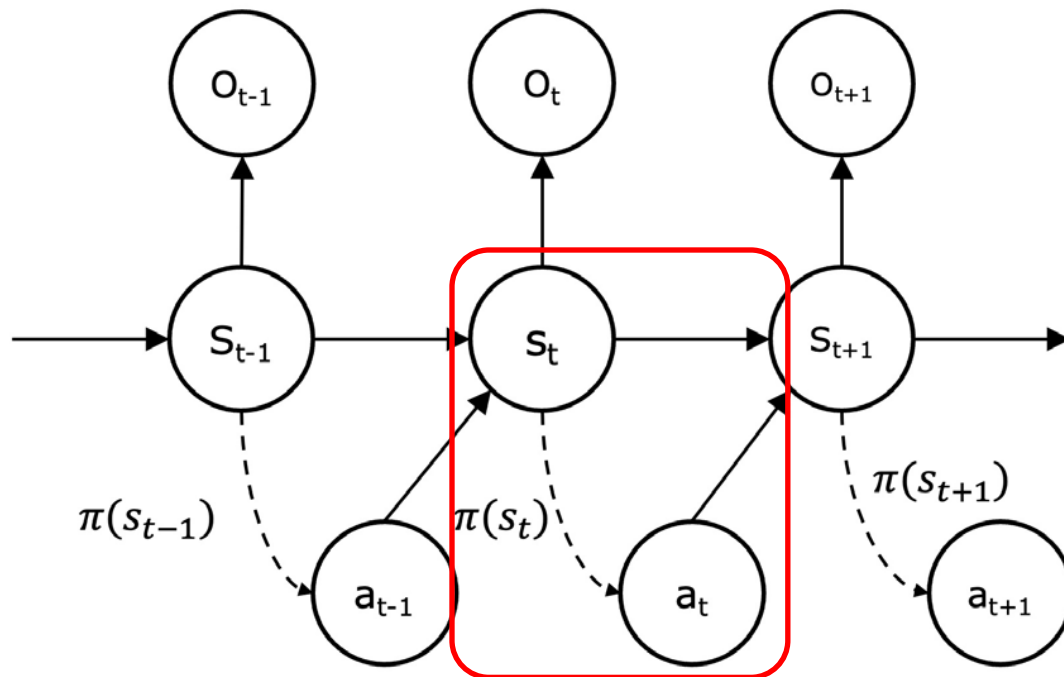
Jianyu Chen (陈建宇)

Institute for Interdisciplinary
Information Sciences

Tsinghua University

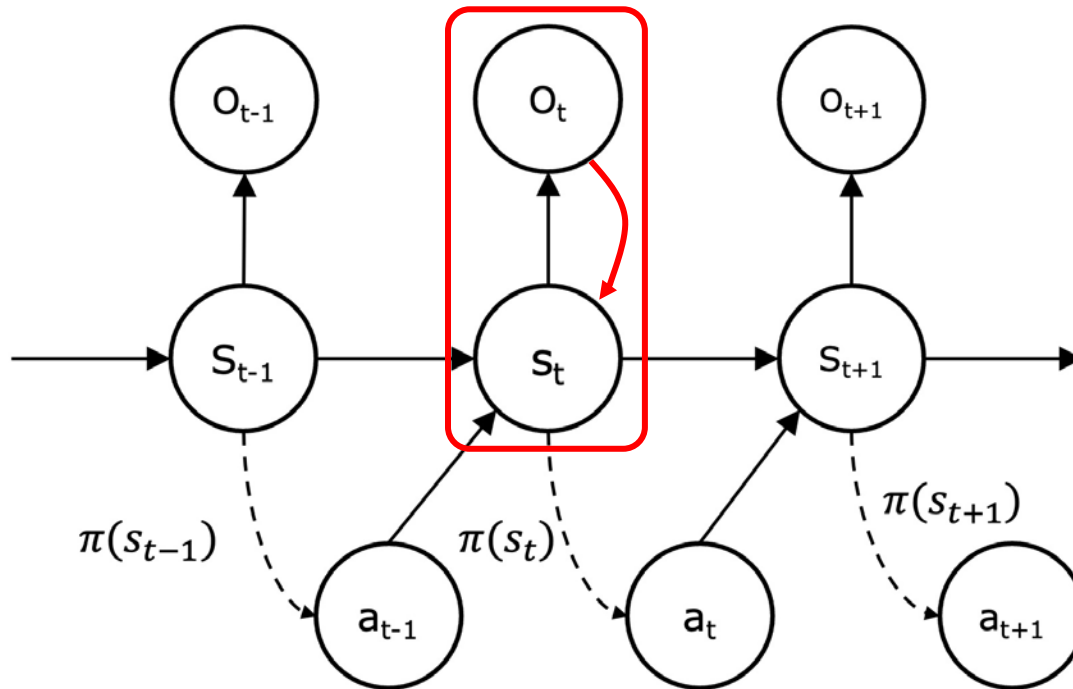
Control

- In previous lectures, we are focusing on **control**, which aims to calculate the action a given the state s
 - PID control
 - Optimal control, MPC
 - Reinforcement learning



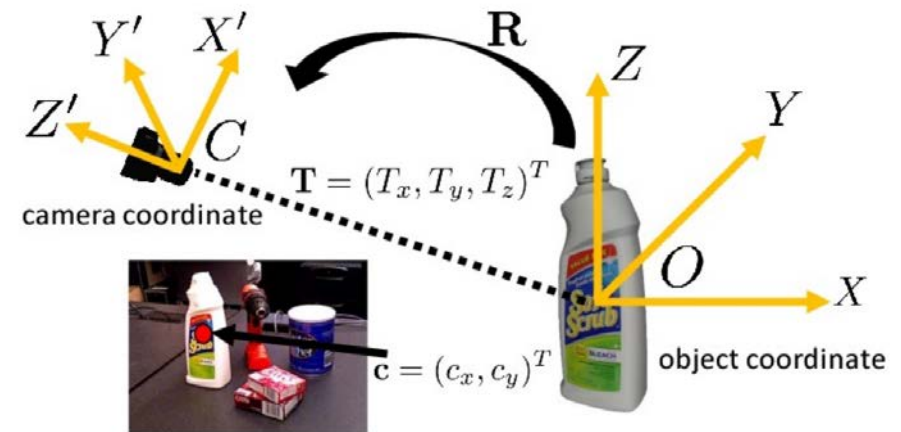
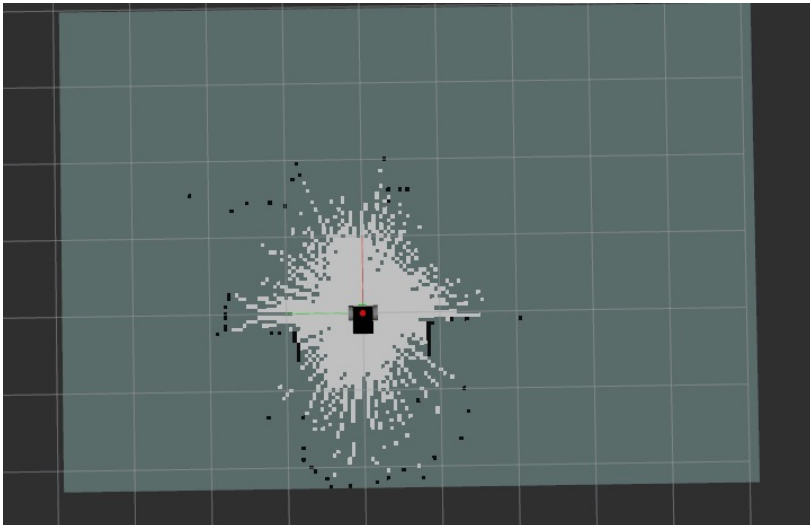
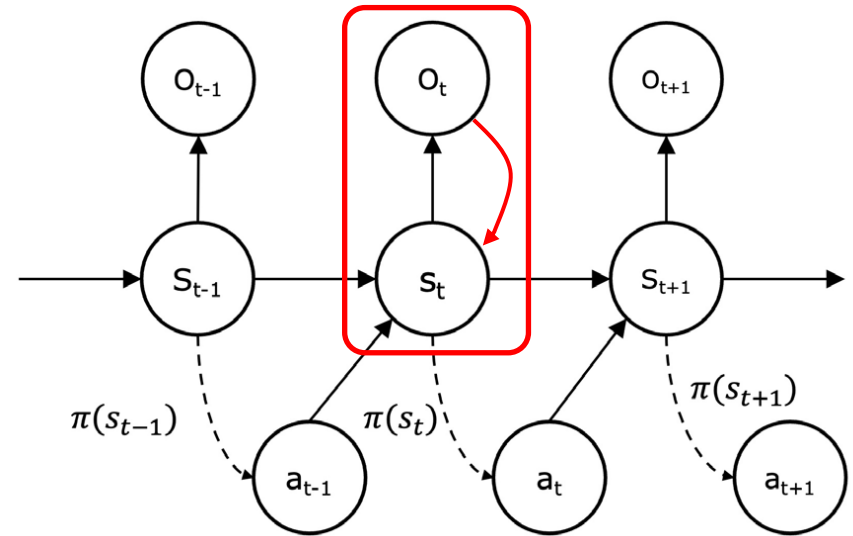
Perception

- However, in the real world, the state s is usually unknown, only some observation o is available
- The goal of **perception** is to calculate the state s given the observation o



Perception

- Estimate proprioceptive
 - Base pose and velocity
- Detect external
 - Object pose



Contents

1

Geometric and Deep Pose Estimation

2

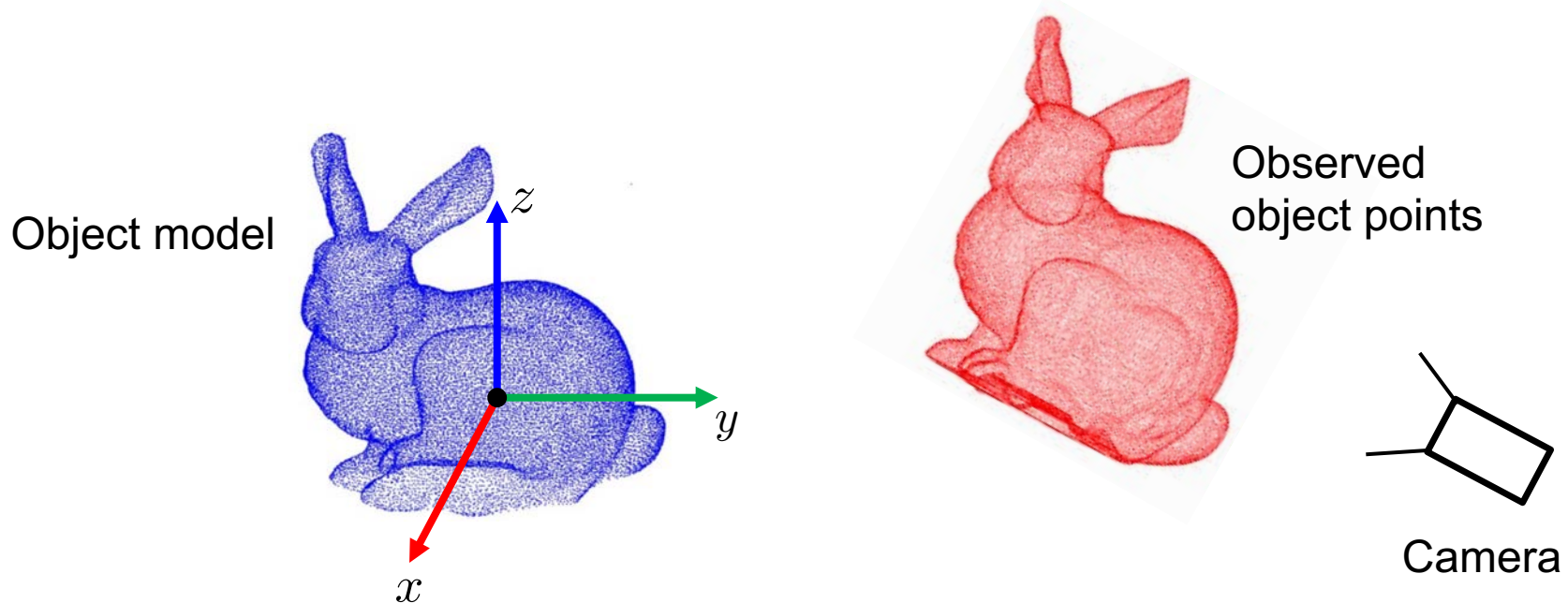
Recursive State Estimation

3

Kalman Filters

Geometric Pose Estimation

- The **geometric pose estimation** problem:
- Known point **relative positions** in object **body frame**
 - Given **observed** object points in the **camera frame**
 - Known **camera pose** in the **world frame**
 - Obtain the **object pose** in the **world frame**



Model Points and Scene Points

- More detailly, let the object frame be $\{o\}$, its points are called the **model points** m_i :

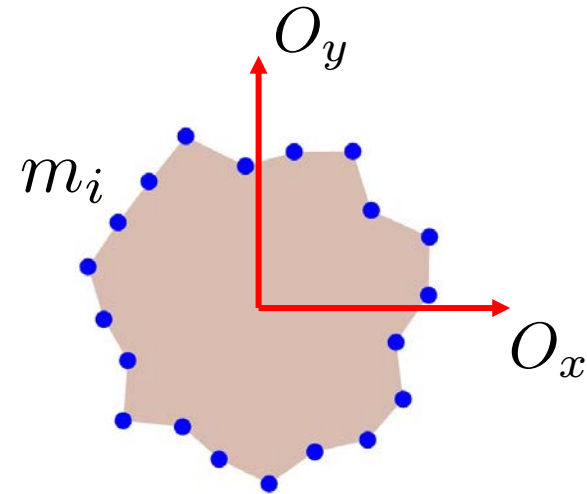
$$p_o^{m_i}$$

- Let the world frame be $\{w\}$, then the object frame $\{o\}$ and camera frame $\{c\}$ in the world frame are:

$$T_{wo} \quad T_{wc}$$

- The camera detects the points of the object in the scene, called the **scene points** s_i , expressed in $\{c\}$:

$$p_c^{s_i}$$



Points Correspondence

- Now assume there are **known correspondences** between the model points and the scene points:

- We know s_i corresponds to m_i

- The i th **scene point** can be expressed in $\{w\}$ as:

$$p_w^{s_i} = T_{wc} p_c^{s_i}$$

- The i th **model point** can be expressed in $\{w\}$ as:

$$p_w^{m_i} = T_{wo} p_o^{m_i}$$

- These two points should be **equal** since they corresponds to one point

$$p_w^{m_i} = T_{wo} p_o^{m_i} = T_{wc} p_c^{s_i} = p_w^{s_i}$$

A Least Square Problem

- Looking at this equation

$$T_{wo}p_o^{m_i} = T_{wc}p_c^{s_i}$$

Unknown Known

- This is like to solve $Ax = b$
- We can write it as a **least-square problem**

$$\min_{T_{wo}} \sum_i ||T_{wo}p_o^{m_i} - T_{wc}p_c^{s_i}||^2$$

Unknown Correspondence

- In practice, we **do not know** the correspondence between the model points and the scene points

$$p_w^{m_i} \quad ? \quad \Rightarrow \quad p_w^{s_i}$$

- How do we get the **correspondence** $j = c_i$, so that s_i corresponds to m_j ?
- If we were **given the pose** of the object, a very straightforward way to define the correspondence is:
 - The corresponding point of model point $p_w^{m_j}$ is just the **closest** scene point $p_w^{s_i}$

Iterative Closest Point (ICP)

□ This idea motivates the **Iterative Closest Point (ICP)** algorithm:

- Start with an **initial guess** of the object pose $\hat{T}_{wo}$
- Find the **closest point correspondence**

$$\forall i, \hat{c}_i = \operatorname{argmin}_{j \in N_m} \|\hat{X}_{wo} p_o^{m_j} - p_w^{s_i}\|^2$$

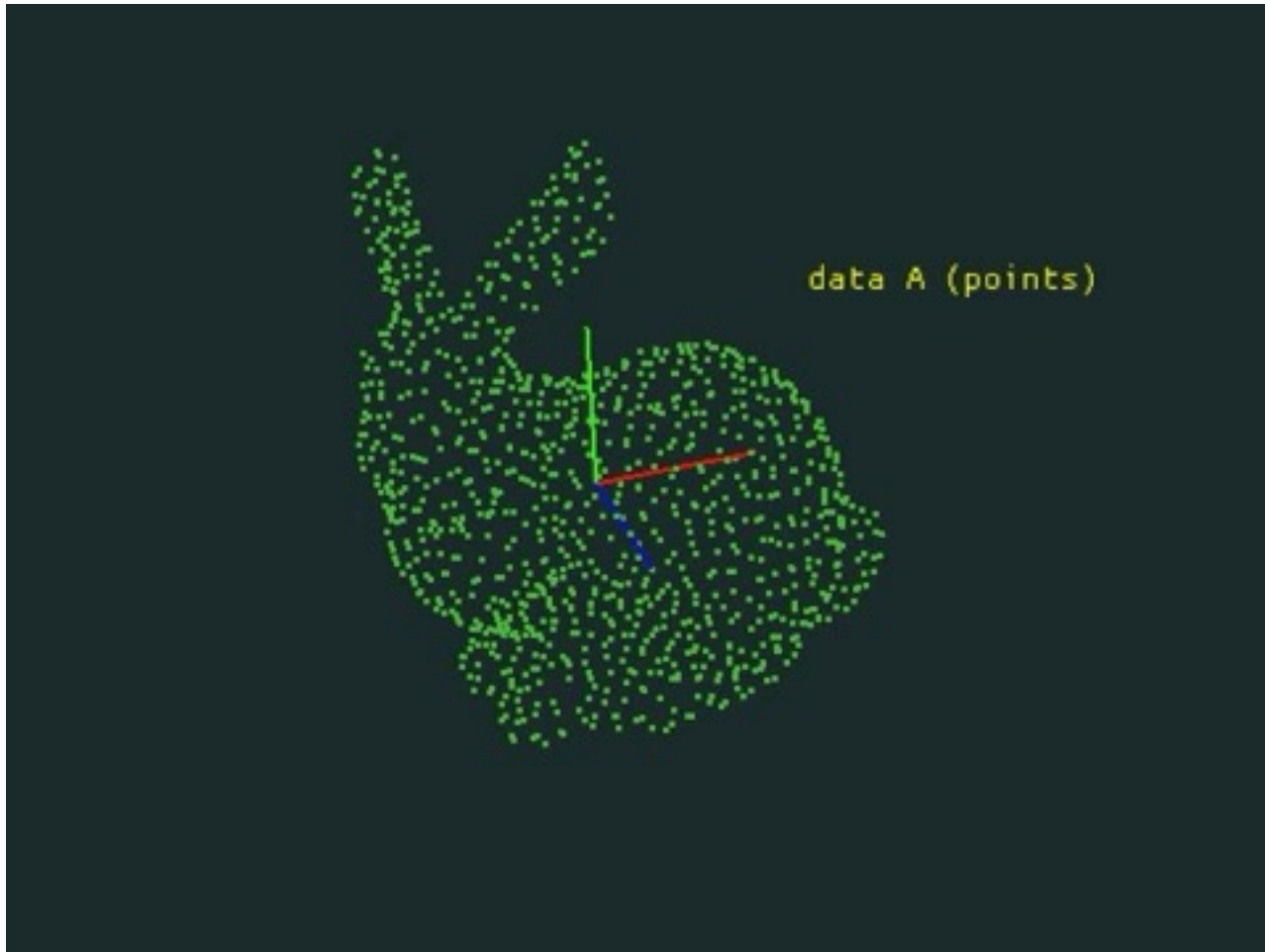
- Solve for the point registration with **known correspondence** to obtain the new pose

$$\hat{T}_{wo} = \min_{T_{wo}} \sum_i \|T_{wo} p_o^{m_{\hat{c}_i}} - T_{wc} p_c^{s_i}\|^2$$

- **Iterate** until convergence

Iterative Closest Point (ICP)

- Point registration for the rabbit



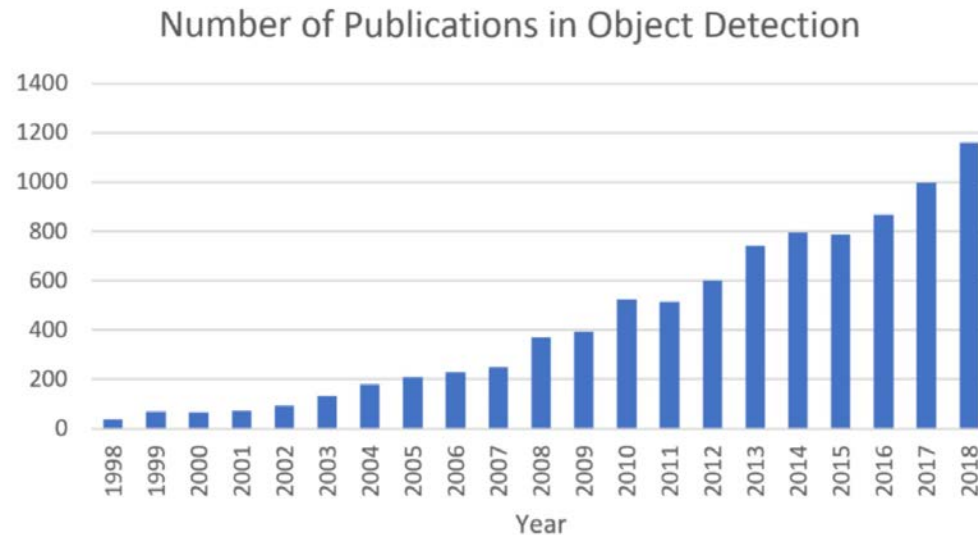
Coherent Point Drift (CPD)

A Framework for Manipulating Deformable Linear
Objects by Coherent Point Drift

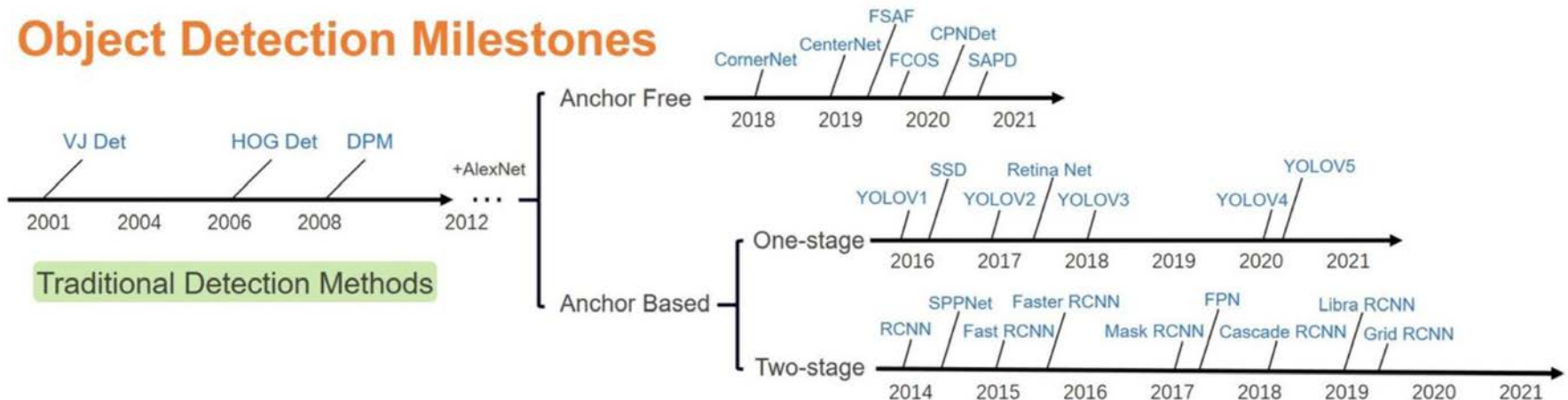
MSC Lab
UC Berkeley
Jan, 2018

Object Detection

- Object detection is an important and popular research area

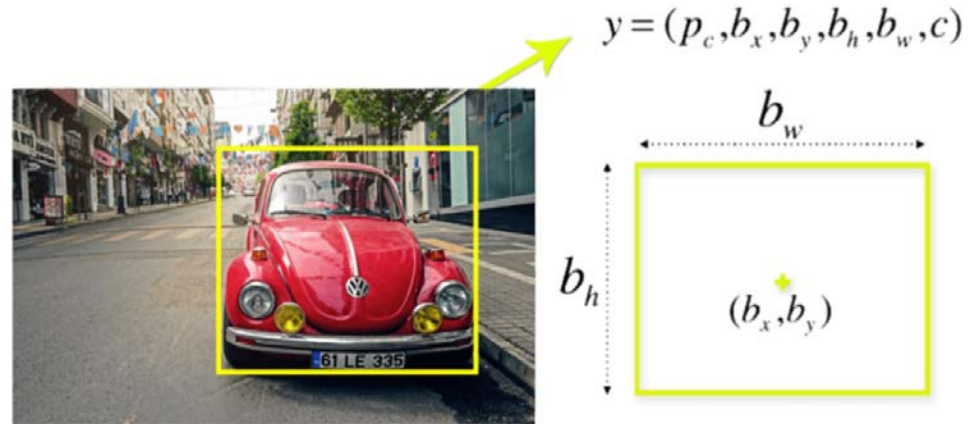


Object Detection Milestones

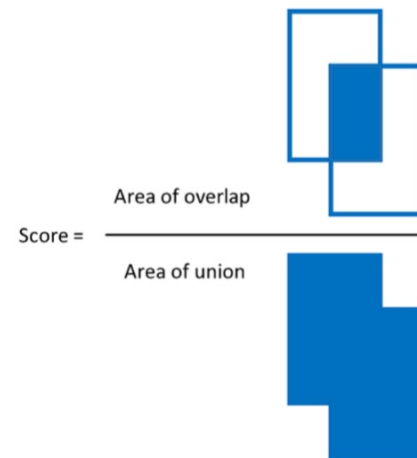


Object Detection

□ Bounding box (bbox)



□ Intersection over Union (IoU)



Mask R-CNN

□ Segmentation/ detection result



Contents

1

Geometric and Deep Pose Estimation

2

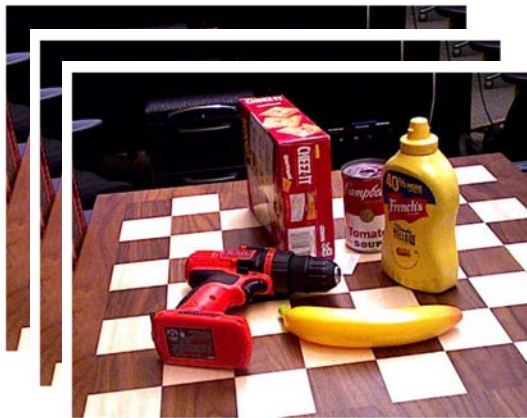
Recursive State Estimation

3

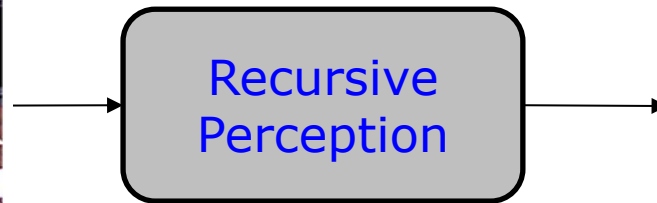
Kalman Filters

From Single Frame to Recursive

- ❑ The above methods perform single frame perception to obtain the object pose information, however
 - They cannot obtain **speed** information
 - Cannot handle occlusion cases
- ❑ Those limitations can be alleviated if we **recursively** take new observations and perform perception considering **historical data**



$z_{1:t}, u_{1:t}$



x_t

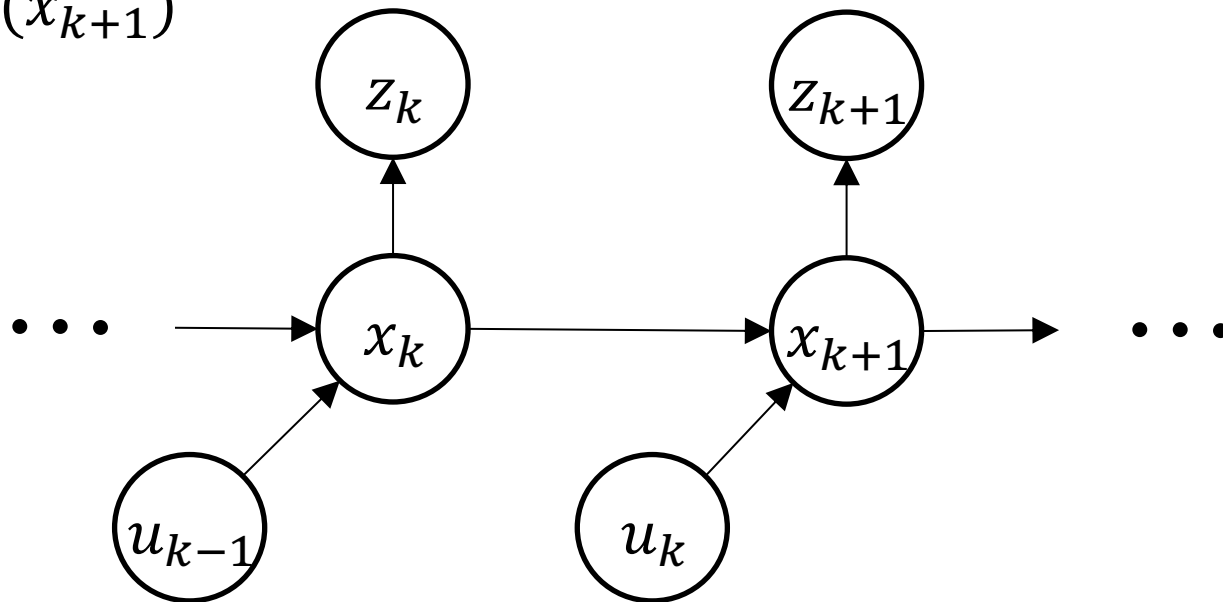
State Estimation Problem

□ Goal:

- Obtain the **posterior** distribution, or the **belief**, of the state x_{k+1} :

$$Bel(x_{k+1}) = P(x_{k+1} | z_0, u_0, \dots, u_k, z_{k+1})$$

- Obtain an **estimation** of x_{k+1} from its belief $Bel(x_{k+1})$



Bayes Solution For State Estimation

□ How to solve for:

$$Bel(x_{k+1}) = P(x_{k+1} | z_0, u_0, \dots, u_k, z_{k+1})$$

□ Use the **Bayes rule**:

$$\begin{aligned} Bel(x_{k+1}) &= P(x_{k+1} | z_0, u_0, \dots, u_k, z_{k+1}) \\ &= \frac{P(z_{k+1} | x_{k+1}, z_0, u_0, \dots, u_k) P(x_{k+1} | z_0, u_0, \dots, u_k)}{P(z_{k+1} | z_0, u_0, \dots, u_k)} \\ &= \eta P(z_{k+1} | x_{k+1}, z_0, u_0, \dots, u_k) P(x_{k+1} | z_0, u_0, \dots, u_k) \end{aligned}$$

□ This is still hard to solve

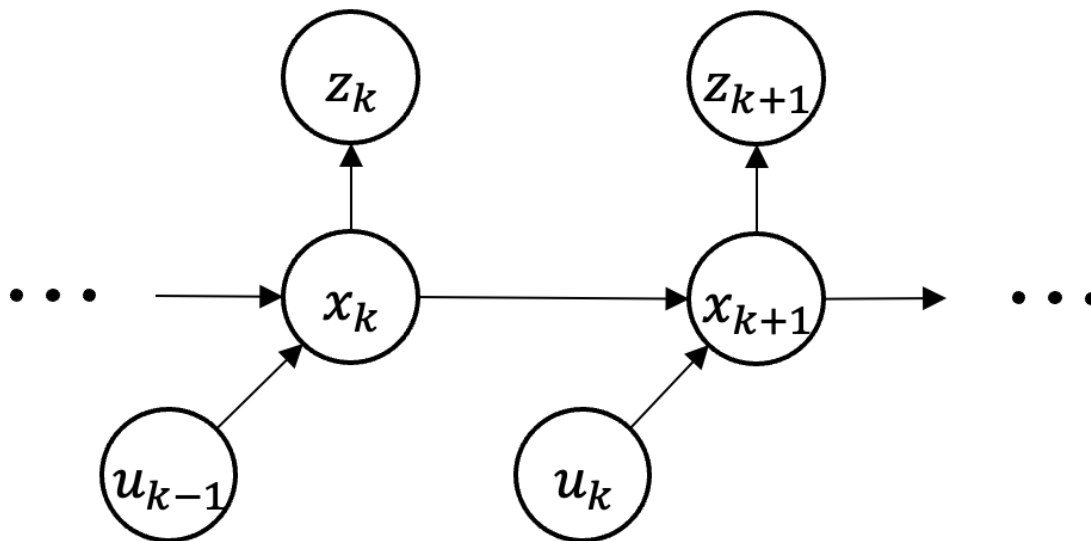
Markov Assumption

- The next state only depends on the **current** state and action (motion model):

$$P(x_{k+1} | x_{0:k}, z_{0:k+1}, u_{0:t}) = P(x_{k+1} | x_k, u_k)$$

- The current observation only depends on the **current** state (observation model):

$$P(z_{k+1} | x_{0:k+1}, z_{0:k}, u_{0:k}) = P(z_{k+1} | x_{k+1})$$



Bayes Filters

$$Bel(x_{k+1}) = P(x_{k+1} | z_0, u_0, \dots, u_k, z_{k+1})$$

$$\text{Bayes} = \eta P(z_{k+1} | x_{k+1}, z_0, u_0, \dots, u_k) P(x_{k+1} | z_0, u_0, \dots, u_k)$$

$$\text{Markov} = \eta P(z_{k+1} | x_{k+1}) P(x_{k+1} | z_0, u_0, \dots, u_k)$$

$$\begin{aligned} \text{Total Prob} &= \eta P(z_{k+1} | x_{k+1}) \int P(x_{k+1} | z_0, u_0, \dots, u_k, x_k) \\ &\quad P(x_k | z_0, u_0, \dots, u_k) dx_k \end{aligned}$$

$$\begin{aligned} \text{Markov} &= \eta P(z_{k+1} | x_{k+1}) \int P(x_{k+1} | u_k, x_k) P(x_k | z_0, u_0, \dots, z_k) dx_k \\ &= \eta P(z_{k+1} | x_{k+1}) \int P(x_{k+1} | u_k, x_k) Bel(x_t) dx_k \end{aligned}$$

Recursive Bayes Filters

$$Bel(x_{k+1}) = \eta P(z_{k+1}|x_{k+1}) \int P(x_{k+1}|u_k, x_k) Bel(x_t) dx_k$$

□ A recursive algorithm for Bayes filters

- Given the current belief $Bel(x_k)$
- Obtain the action applied u_k , and calculate the **prior distribution** of x_k :

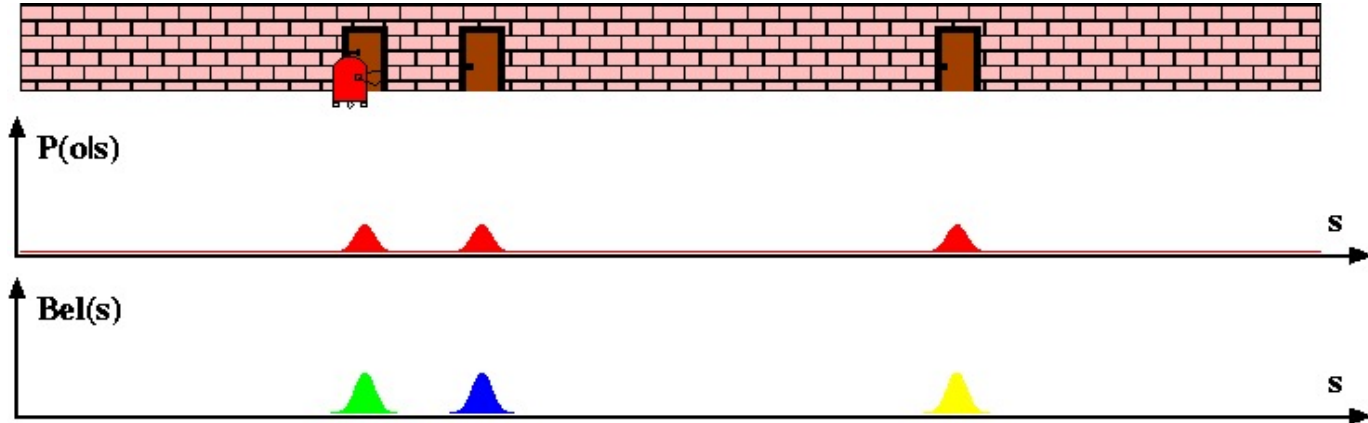
$$\overline{Bel}(x_{k+1}) = \int P(x_{k+1}|u_k, x_k) Bel(x_k) dx_k \quad \forall x_{k+1}$$

- Get a new observation z_t and calculate the **posterior distribution** of x_t :

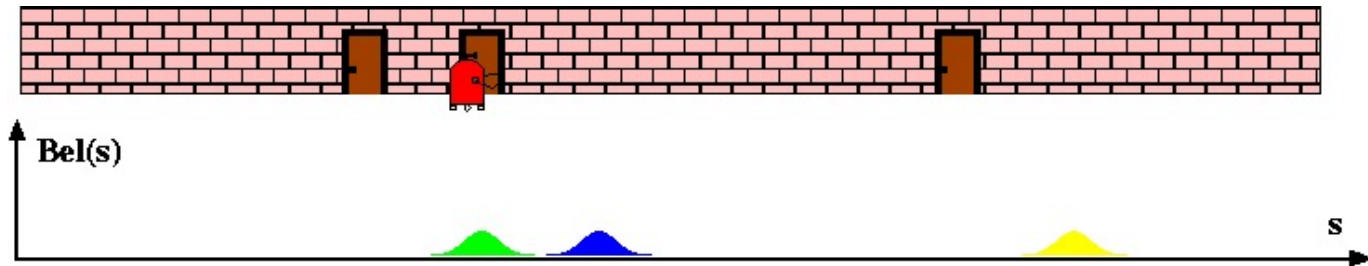
$$Bel(x_{k+1}) = \eta P(z_{k+1}|x_{k+1}) \overline{Bel}(x_{k+1}) \quad \forall x_{k+1}$$

Example

□ Localization by sensing the doors



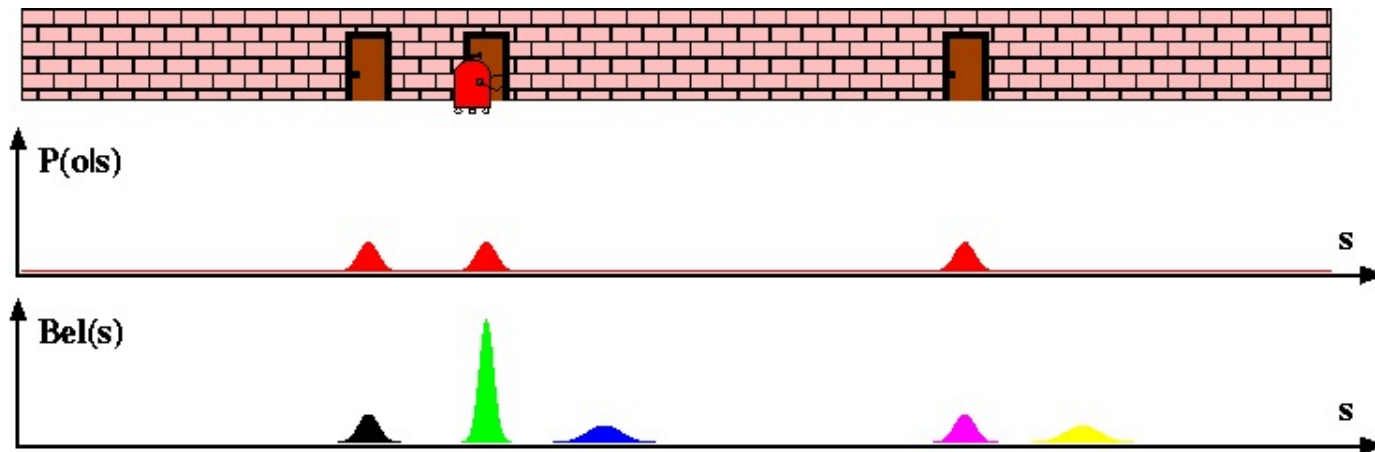
$$Bel(x_{k+1}) = \eta P(z_{k+1}|x_{k+1})\overline{Bel}(x_{k+1}) \quad \forall x_{k+1}$$



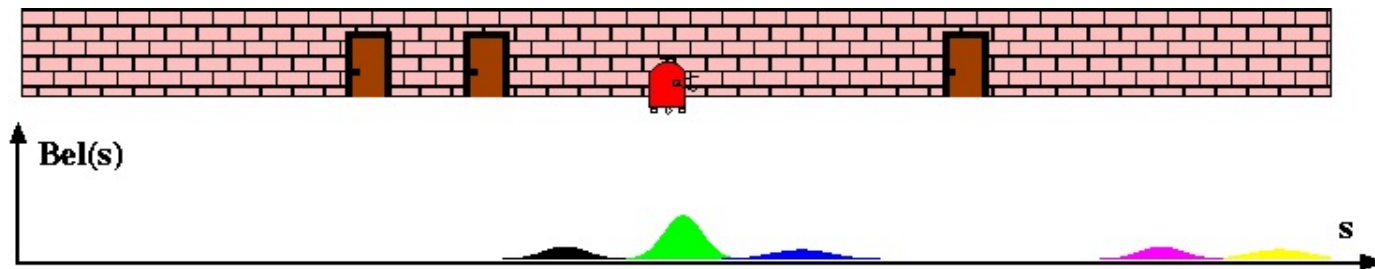
$$\overline{Bel}(x_{k+1}) = \int P(x_{k+1}|u_k, x_k) Bel(x_k) dx_k \quad \forall x_{k+1}$$

Example

□ Localization by sensing the doors



$$Bel(x_{k+1}) = \eta P(z_{k+1}|x_{k+1}) \overline{Bel}(x_{k+1}) \quad \forall x_{k+1}$$



$$\overline{Bel}(x_{k+1}) = \int P(x_{k+1}|u_k, x_k) Bel(x_k) dx_k \quad \forall x_{k+1}$$

Obtain Estimation From Distribution

□ The above method obtains the **posterior distribution** of the state, while **estimation** requires obtaining a **single value** from the distribution

□ Maximum A Posterior (MAP)

$$f(x|z) = \frac{f(z|x) f(x)}{f(z)}$$

$$\hat{x}^{\text{MAP}} := \arg \max_{x \in \mathcal{X}} f(x|z) = \arg \max_{x \in \mathcal{X}} f(z|x) f(x)$$

□ Minimum Mean Squared Error (MMSE)

$$\begin{aligned} \hat{x}^{\text{MMSE}} &:= \arg \min_{\hat{x} \in \mathbb{R}^N} \mathbb{E} [(\hat{x} - x)^T (\hat{x} - x) | z] \\ &= \arg \min_{\hat{x} \in \mathbb{R}^N} (\hat{x}^T \hat{x} - 2\hat{x}^T \mathbb{E} [x|z] + \mathbb{E} [x^T x | z]) \end{aligned}$$

Obtain Estimation From Distribution

- Minimum Mean Squared Error (**MMSE**) is the **mean**

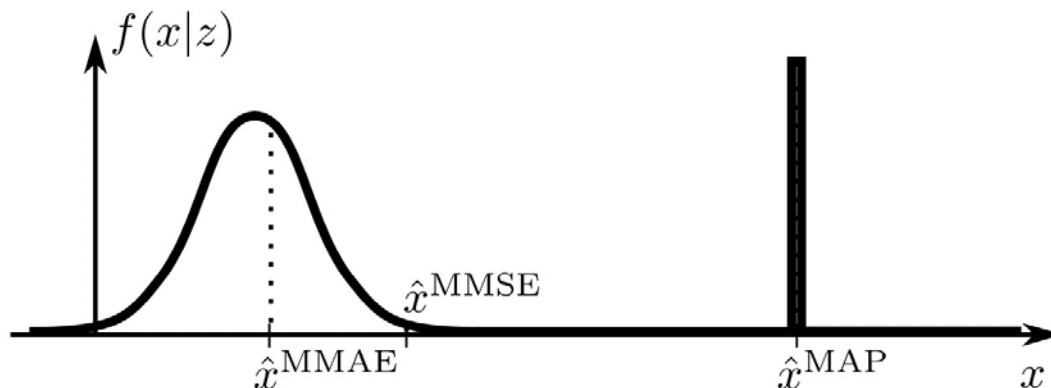
$$\hat{x}^{\text{MMSE}} = \text{E} [x|z]$$

- Minimum Mean Absolute Error (**MMAE**) is the **median**

$$\hat{x}^{\text{MMAE}} := \arg \min_{\hat{x} \in \mathbb{R}^N} \text{E} [|\hat{x} - x| | z]$$

$$\Pr (x \leq \hat{x}^{\text{MMAE}} | z) = \Pr (x > \hat{x}^{\text{MMAE}} | z) = \frac{1}{2}$$

- Comparison of different estimation



Contents

1

Geometric and Deep Pose Estimation

2

Recursive State Estimation

3

Kalman Filters

Recursive Bayes Filters

$$Bel(x_{k+1}) = \eta P(z_{k+1}|x_{k+1}) \int P(x_{k+1}|u_k, x_k) Bel(x_t) dx_k$$

□ A recursive algorithm for Bayes filters

- Given the current belief $Bel(x_k)$
- Obtain the action applied u_k , and calculate the **prior distribution** of x_k :

$$\overline{Bel}(x_{k+1}) = \int P(x_{k+1}|u_k, x_k) Bel(x_k) dx_k \quad \forall x_{k+1}$$

- Get a new observation z_t and calculate the **posterior distribution** of x_t :

$$Bel(x_{k+1}) = \eta P(z_{k+1}|x_{k+1}) \overline{Bel}(x_{k+1}) \quad \forall x_{k+1}$$

From Discrete To Continuous

- **Ideal** Bayes Filter works in general **discrete** world

$$\overline{Bel}(x_{k+1}) = \int P(x_{k+1}|u_k, x_k) Bel(x_k) dx_k \quad \forall x_{k+1}$$

Explicit summation

$$Bel(x_{k+1}) = \eta P(z_{k+1}|x_{k+1}) \overline{Bel}(x_{k+1}) \quad \forall x_{k+1}$$

Explicit normalization

- How about **continuous** random variables?
- **Kalman Filter** is an exact implementation of Bayes Filter in **Linear Gaussian** case

Linear Gaussian Systems

□ Motion model

$$x_{k+1} = Ax_k + Bu_k + w_k$$

□ Sensor model

$$z_{k+1} = Cx_{k+1} + v_{k+1}$$

□ Notations

- $A \in \mathbb{R}^{n \times n}$ $B \in \mathbb{R}^{n \times m}$ $C \in \mathbb{R}^{k \times n}$ are matrices describing the motion and sensor model
- $w_k \sim N(0, W)$ is the **process noise** where $W \in \mathbb{R}^{n \times n}$ is its covariance
- $v_{k+1} \sim N(0, V)$ is the **measurement noise** where $V \in \mathbb{R}^{k \times k}$ is its covariance

Kalman Filter Algorithm

□ Algorithm **Kalman_Filter**($\mu_k, \Sigma_k, u_k, z_{k+1}$):

■ **Prior update:**

$$\begin{aligned}\bar{\mu}_{k+1} &= A\mu_k + Bu_k \\ \bar{\Sigma}_{k+1} &= A\Sigma_k A^T + W\end{aligned}$$

■ **Measurement update:**

$$\begin{aligned}K_{k+1} &= \bar{\Sigma}_{k+1} C^T (C \bar{\Sigma}_{k+1} C^T + V)^{-1} \\ \mu_{k+1} &= \bar{\mu}_{k+1} + K_{k+1} (z_{k+1} - C \bar{\mu}_{k+1}) \\ \Sigma_{k+1} &= (I - K_{k+1} C) \bar{\Sigma}_{k+1}\end{aligned}$$

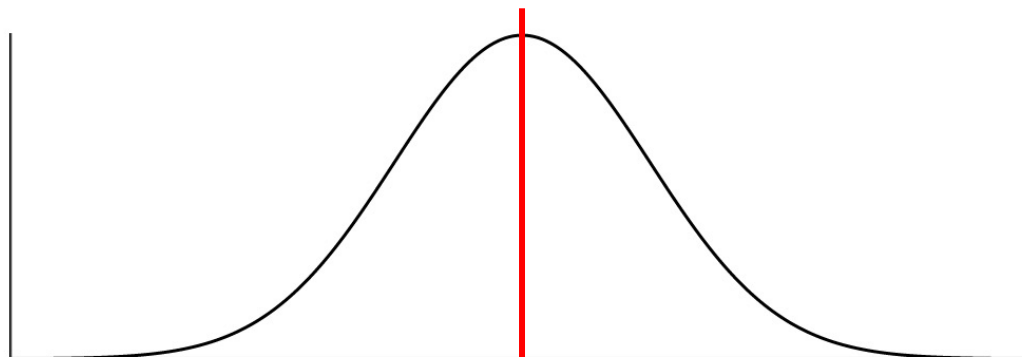
■ **Return:**

$$\mu_{k+1}, \Sigma_{k+1}$$

Extract Estimates For Kalman Filter

- ❑ The Kalman Filter algorithm returns the updated **Gaussian belief distribution** μ_t, Σ_t
- ❑ Estimation requires extracting **statistic value** from the belief distribution
- ❑ What statistics should we extract for Kalman Filter? E.g, MAP, MMSE, or MMAE?
- ❑ For Gaussian distribution, they are actually the **same!**

$$\hat{x}_t = \hat{x}_t^{\text{MAP}} = \hat{x}_t^{\text{MMSE}} = \hat{x}_t^{\text{MMAE}} = \mu_t$$



Kalman Filter In Non-Gaussian Systems

- Does Kalman filter must be applied to linear Gaussian systems?
 - No, it can be applied to linear systems with **non-Gaussian** noises
 - Only the **mean** and **covariance** information is used in the Kalman filter algorithm
- What property does it have when applying to non-Gaussian problems?
 - It's **not** the optimal filter now
 - But is best among all **linear estimators**

$$\hat{x}_k = \bar{\mu}_k + K(z_k - C\bar{\mu}_k)$$

Estimation For Nonlinear Systems

- What if the system models are nonlinear?

$$x_{k+1} = g(x_k, u_k) + w_k$$

$$z_{k+1} = h(x_{k+1}) + v_{k+1}$$

- If g and h are differentiable, we can use their gradient information

$$g'(x_k, u_k) = \frac{\partial g(x_k, u_k)}{\partial x_k}$$

$$h'(x_k) = \frac{\partial h(x_k)}{\partial x_k}$$

Estimation For Nonlinear Systems

- Approximate the nonlinear function with first order Taylor expansion

- Motion model

$$\begin{aligned} g(u_k, x_k) &\approx g(u_k, \mu_k) + g'(u_k, \mu_k)(x_k - \mu_k) \\ &= g(u_k, \mu_k) + A(x_k - \mu_k) \end{aligned}$$

- Sensor model

$$\begin{aligned} h(x_k) &\approx h(\bar{\mu}_k) + h'(\bar{\mu}_k)(x_k - \bar{\mu}_k) \\ &= h(\bar{\mu}_k) + C(x_k - \bar{\mu}_k) \end{aligned}$$

Extended Kalman Filter Algorithm

□ Algorithm **Extended Kalman_Filter**(μ_k , Σ_k , u_k , z_{k+1}):

■ **Prior update:**

$$\bar{\mu}_{k+1} = \underline{g(\mu_k, u_k)}$$

$$\bar{\Sigma}_{k+1} = A\Sigma_k A^T + W$$

■ **Measurement update:**

$$K_{k+1} = \bar{\Sigma}_{k+1} C^T (C\bar{\Sigma}_{k+1} C^T + V)^{-1}$$

$$\mu_{k+1} = \bar{\mu}_{k+1} + K_{k+1} (z_{k+1} - \underline{h(\bar{\mu}_{k+1})})$$

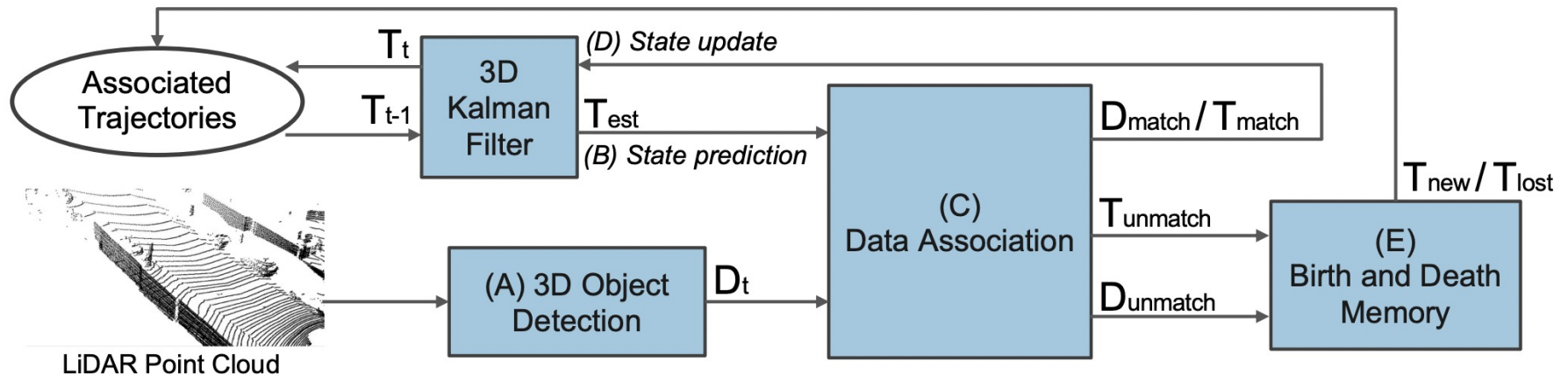
$$\Sigma_{k+1} = (I - K_{k+1} C) \bar{\Sigma}_{k+1}$$

■ **Return:**

$$\mu_{k+1}, \Sigma_{k+1}$$

Application: Object Tracking

- ❑ Learning-based single frame pose estimation
- ❑ EKF for recursive state estimation



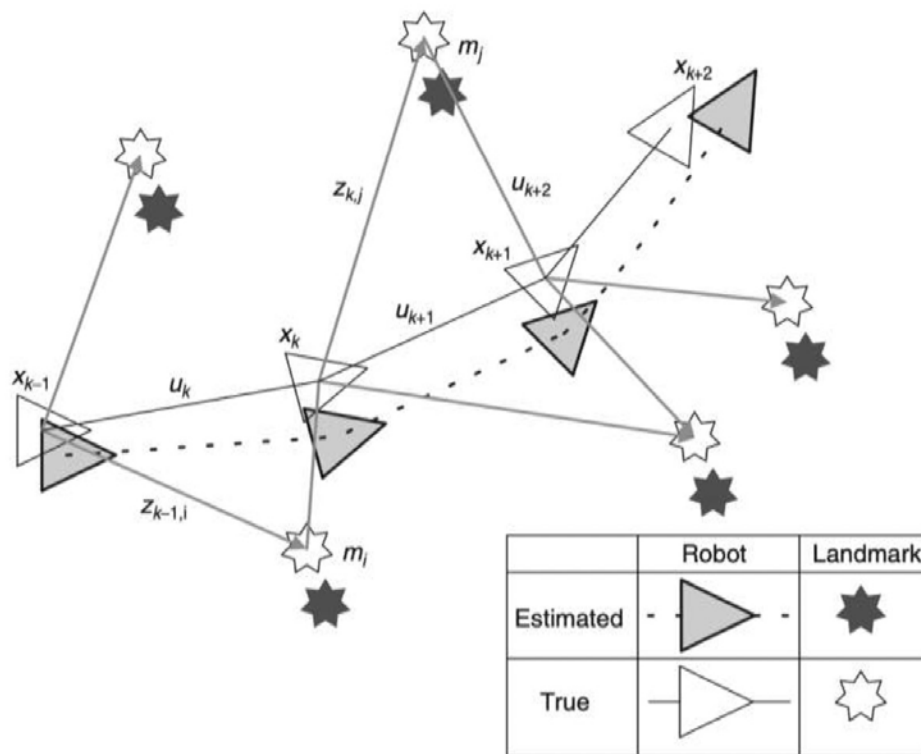
Qualitative Results

Application: EKFSLAM

□ Use EKF for **Simultaneous Localization and Mapping (SLAM)**

□ Need to estimate both the robot **pose state** and the **landmark**:

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$$



Application: MonoSLAM

- Use single camera to extract landmarks, then EKFSLAM

Real-Time
Camera Tracking
in Unknown Scenes

Application: ROVIO

- Combine inertial sensor information

ROVIO: Robust Visual Inertial Odometry Using a Direct EKF-Based Approach

<http://github.com/ethz-asl/rovio>

Michael Bloesch, Sammy Omari, Marco Hutter, Roland Siegwart



Autonomous Systems Lab

ETH zürich

Thank you!



清华大学 交叉信息研究院
Institute for Interdisciplinary Information Sciences, Tsinghua University

